

Messages > Building with Claude

Using the Messages API

 Copy page 

Practical patterns and examples for using the Messages API effectively

Anthropic offers two ways to build with Claude, each suited to different use cases:

	Messages API	Claude Managed Agents
What it is	Direct model prompting access	Pre-built, configurable agent harness that runs in managed infrastructure
Best for	Custom agent loops and fine-grained control	Long-running tasks and asynchronous work

This guide covers common patterns for working with the Messages API, including basic requests, multi-turn conversations, prefill techniques, and vision capabilities. For complete API specifications, see the [Messages API reference](#). For the managed agent harness instead, see the [Claude Managed Agents overview](#).

 To learn how zero data retention (ZDR) applies to this feature, see [API and data retention](#).

Basic request and response

 The `temperature`, `top_p`, and `top_k` sampling parameters are not supported on Claude 4.7 and later models and Claude Mythos Preview. Setting them to a non-default value returns a 400 error. Omit them from request payloads and use prompting to guide the model's behavior instead. See the [migration guide](#).

```
message = anthropic.Anthropic().message(
    model="claude-opus-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Hello!"}]
)
print(message)
```

Output

```
{
  "id": "msg_01XFDUDYJgAACzvnptvVoYEL",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Hello!"
    }
  ],
  "model": "claude-opus-5",
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 12,
    "output_tokens": 6
  }
}
```

Refusal responses (`stop_reason: "refusal"`) also include a `stop_details` object identifying the policy category that triggered the refusal, on every model. See [Handling stop reasons](#) for the field reference and example handling code.

Multiple conversational turns

The Messages API is stateless, which means that you always send the full conversational history to the API. You can use this pattern to build up a conversation over time. Earlier conversational turns don't necessarily need to actually originate from Claude. You can use synthetic `assistant` messages.

```
message = anthropic.Anthropic().message(
    model="claude-opus-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "Hello!"},
        {"role": "assistant", "content": "Hi there!"},
        {"role": "user", "content": "How are you?"},
    ],
)
print(message)
```

```
Output
{
  "id": "msg_018gCsTGsXkYJVqYPxTgDHBu",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Sure, I'd be happy to"
    }
  ],
  "model": "claude-opus-5",
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 30,
    "output_tokens": 309
  }
}
```

System role in messages

On Claude Fable 5.1, [Claude Mythos 5.1](#), Claude Fable 5, [Claude Mythos 5](#), Claude Opus 4.8, and Claude Opus 5, you can include messages with `"role": "system"`

after a user turn (subject to [placement rules](#)) to add a new system instruction partway through a conversation. A `system` message cannot be the first entry in

Claude Platform Docs

[Messages](#)[Managed Agents](#)[Admin](#)[Resources](#) ▾

{ } API r

 K

First steps

[Intro to Claude](#)[Get your API key](#)[Quickstart](#)[Authentication](#)

Building with Claude

[Features overview](#)[Using the Messages API](#)[Stop reasons and fallback](#)[Refusals and fallback](#)[Fallback credit](#)

Model capabilities

[Effort](#)[Task budgets \(beta\)](#)[Fast mode \(research preview\)](#)[Structured outputs](#)**Béatrice**

Admin · Béatrice's I...



A mid-conversation system message has the same authority as the top-level `system` field, but because it is appended to the end of the message history, it does not invalidate any cached prefix that came before it. Use the top-level `system` field for instructions that should apply from the very first turn, and a mid-conversation system message for instructions that only become relevant later.

See [Mid-conversation system messages](#) for the complete guide, including how to combine it with [prompt caching](#).

Prefilling Claude's response

You can pre-fill part of Claude's response in the last position of the input messages list. Use this technique to shape Claude's response. The following example uses `"max_tokens": 1` to get a single multiple choice answer from Claude.

⚠ Prefilling is not supported on Claude 4.6 and later models and [Claude Mythos Preview](#). Requests using prefill with these models return a 400 error. Use [structured outputs](#) on models that support it, or system prompt instructions, instead. See the [migration guide](#) for migration patterns.

[Ask Docs](#) 

```
message = anthropic.Anthropic().message(
    model="claude-sonnet-4-5",
    max_tokens=1,
    messages=[
        {
            "role": "user",
            "content": "What is latin"
        },
        {"role": "assistant", "content": "C"}
    ],
)
print(message)
```

Output

```
{
  "id": "msg_01Q8Faay6S7QPTvEUUQARt7h",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "C"
    }
  ],
  "model": "claude-sonnet-4-5",
  "stop_reason": "max_tokens",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 42,
    "output_tokens": 1
  }
}
```

Vision

Claude can read both text and images in requests. You can supply images using the `base64`, `url`, or `file` source types. The `file` source type references an image uploaded through the [Files API](#). Supported media types are `image/jpeg`, `image/png`, `image/gif`, and `image/webp`. See the [vision guide](#) for more details.

```
import base64
import httpx2

# Option 1: Base64-encoded image
image_url = "https://platform.claude.
image_media_type = "image/jpeg"
image_data = base64.standard_b64encod

message = anthropic.Anthropic().messa
    model="claude-opus-5",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "base
                        "media_type":
                        "data": image
                    },
                },
                {"type": "text", "tex
            ],
        }
    ],
)
print(message)

# Option 2: URL-referenced image
message_from_url = anthropic.Anthropi
    model="claude-opus-5",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {
                        "type": "url"
                        "url": "https
                    },
                },
                {"type": "text", "tex
            ],
        }
    ],
)
print(message_from_url)
```

Output



```
{
  "id": "msg_011CdKmWtV3oFx1C5yUbf5CY",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "This image is a beauti"
    }
  ],
  "model": "claude-opus-5",
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 1030,
    "output_tokens": 350
  }
}
```

Next steps



Stop reasons and fallback

Handle each
`stop_reason...`



Tool use with Claude

Give Claude tools
to call external...



Computer use tool

Control desktop
computer...



Browser use tool

Let Claude
navigate, read,...



Structured outputs

Get guaranteed,
schema...

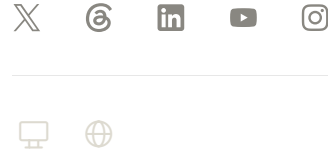


Task budgets

Set an advisory token
across a full agentic

Was this page helpful?





AI Agents
Code modernization
Coding
Customer support
Financial services
Government
Higher education
K-12 teachers
Life sciences
Partners
Claude on AWS
Claude on Google Cloud

Anthropic
Big data
Career courses
Economic future
Research
Security
and anthropic
Compliance
Transparency
Service partners
Startups
Program

Help and security
Availability
Status
Support
Connectors
Discord
Customer stories
Engineering
Policies
Privacy
Events
Policy
Responsible disclosure
Claude policy
Terms of service:
Commercial
Terms of service:
Consumer Usage
policy